

Spark – Guía completa de arquitectura y optimización avanzada

Manual técnico en español (España) – Arquitectura, ejecución y rendimiento (Spark SQL & PySpark).

Índice de contenidos

1. Arquitectura de Spark
2. Ciclo de vida de una query
3. Modelo de computación distribuida
4. Data Skew (definición, diagnóstico y soluciones)
5. Shuffle, Joins y Broadcast
6. Particiones, paralelismo y memoria
7. PySpark en producción (buenas prácticas)
8. Delta Lake (OPTIMIZE, Z-ORDER, VACUUM, ANALYZE)
9. Photon vs JVM/GC
10. Autoscaling vs Single Node (coste y rendimiento)
11. AQE (Adaptive Query Execution)
12. Playbooks y Checklists
13. Anexo: Chuleta Spark Master (comandos y parámetros)

1. Arquitectura de Spark

Spark se basa en una arquitectura distribuida con un nodo Driver que coordina la ejecución, un Cluster Manager que asigna recursos, y múltiples nodos Worker que ejecutan procesos Executor. Cada Executor ejecuta tasks que procesan particiones de datos.

Componentes principales

Componente	Rol
Driver	Coordina la ejecución, construye los planes lógico/físico y envía tasks a los executors.
Cluster Manager	YARN / Kubernetes / Databricks / Standalone: asigna CPU y memoria.
Worker Node	Servidor/VM que aloja uno o varios executors.
Executor	Proceso que ejecuta tasks y mantiene cache local.
Task	Unidad mínima de ejecución (una partición).
Stage	Conjunto de tasks sin dependencia de shuffle.
Job	Conjunto de stages disparado por una acción (count, write, collect, show).

Flujo básico

```
# Ejemplo conceptual en Databricks
df1 = spark.read.csv('dbfs:/mnt/landing/fact.csv', header=True)
df2 = spark.read.csv('dbfs:/mnt/landing/dim.csv', header=True)
res = df1.join(df2, 'id').filter("country = 'ES'")
res.write.mode('overwrite').parquet('dbfs:/mnt/curated/out')
```

2. Ciclo de vida de una query

El ciclo de vida de una query en Spark sigue estas fases: 1) Logical Plan → 2) Optimized Logical Plan (Catalyst) → 3) Physical Plan → 4) Execution Plan (Tungsten o Photon) → 5) Runtime (AQE).

```
EXPLAIN FORMATTED
SELECT /*+ BROADCAST(d) */ f.*
FROM fact f JOIN dim d ON f.k = d.k
WHERE f.dt >= DATE'2025-10-01';
```

3. Modelo de computación distribuida

Spark divide los datasets en particiones. Cada partición se procesa por una task en un executor. Las transformaciones narrow no requieren shuffle; las wide (joins, groupBy, orderBy) sí, introduciendo límites de stage y movimiento de datos por red.

Regla práctica: tamaño objetivo de archivo 128–512 MB; número de particiones de shuffle ≈ núcleos totales del cluster × 2–3.

4. Data Skew: definición, diagnóstico y soluciones

Data skew es el desbalance extremo de datos por clave/partición. Provoca que algunas tasks tarden mucho más que otras, ya que procesan muchas más filas. El job finaliza cuando termina la task más lenta.

Síntomas y detección

```
-- Top claves por volumen (Spark SQL)
SELECT customer_id, COUNT(*) AS c
FROM t
GROUP BY customer_id
ORDER BY c DESC
LIMIT 10;
```

```
-- En PySpark
df.groupBy("customer_id").count().orderBy(F.desc("count")).show(10)
```

Soluciones (en orden de impacto)

1) AQE Skew Join Handling: habilita la partición adaptativa y el manejo automático de skew.

```
SET spark.sql.adaptive.enabled = true;
SET spark.sql.adaptive.skewJoin.enabled = true;
SET spark.sql.adaptive.skewedPartitionThresholdInBytes = 268435456; -- 256 MB
```

2) Reparticionar por clave: distribuye homogéneamente antes de joins/aggregations.

```
df = df.repartition(192, "customer_id")
```

3) Salting: rompe claves calientes inyectando sales aleatorias y replicando el lado pequeño.

```
import pyspark.sql.functions as F

big_s = big.withColumn("k_s", F.concat_ws("#", "k", (F.rand() * 8).cast("int")))

small_s = (small
    .withColumn("salt", F.explode(F.array(*[F.lit(i) for i in range(8)])))
    .withColumn("k_s", F.concat_ws("#", "k", F.col("salt")))
    .drop("salt"))

res = big_s.join(small_s, "k_s")
```

4) Pre-agregación: reduce el volumen de datos antes del join.

```
df_pre = fact.groupBy("k").agg(F.sum("amount").alias("sum_amt"))
```

5) Separar outliers: procesa claves VIP o extremadamente grandes en un flujo aparte.

6) Particionamiento/Z-ORDER: almacena físicamente por clave y usa Z-ORDER para data skipping fino.

5. Shuffle, Joins y Broadcast

Los shuffles son operaciones caras que mueven datos entre nodos. Se producen en joins, groupBy, distinct, orderBy, repartition. Reduce shuffles aplicando filtros y proyecciones temprano y usando broadcast cuando un lado es pequeño.

```
-- Configuración de broadcast en Spark SQL
SET spark.sql.autoBroadcastJoinThreshold = 200MB;
SET spark.sql.broadcastTimeout = 600;
```

```
-- Hint de broadcast en SQL
SELECT /*+ BROADCAST(dim) */ *
FROM fact f
JOIN dim ON f.k = dim.k;
```

```
# Ejemplo en PySpark (Databricks / Delta)
import pyspark.sql.functions as F
```

```
dim = spark.read.format("delta").load("dbfs:/mnt/curated/dim_customer")
fact = (spark.read.format("delta")
    .load("dbfs:/mnt/bronze/fact_sales")
    .repartition(192, "customer_id"))
```

```
res = fact.join(F.broadcast(dim), "customer_id")
```

6. Particiones, paralelismo y memoria

```
SET spark.sql.shuffle.partitions = 192;
SET spark.sql.files.maxPartitionBytes = 134217728; -- 128 MB por partición de lectura
```

Reglas rápidas de particionado

- Objetivo de tamaño por archivo: 128–512 MB.
- `spark.sql.shuffle.partitions` $\approx$ núcleos totales $\times$ 2–3.
- Usa `repartition(n, col)` para balancear joins/agg; `coalesce(n)` para reducir sin shuffle.

7. PySpark en producción: buenas prácticas

Evita UDFs de Python cuando existan funciones nativas

```
# Mal: UDF Python fila a fila
from pyspark.sql.functions import udf

@udf("double")
def my_udf(x):
    return x * 1.21

df = df.withColumn("price_vat", my_udf("price"))

# Mejor: expresión nativa
import pyspark.sql.functions as F
df = df.withColumn("price_vat", F.col("price") * F.lit(1.21))
```

pandas UDF vectorizadas

```
from pyspark.sql.functions import pandas_udf
import pandas as pd

@pandas_udf("double")
def p95(v: pd.Series) -> float:
    return float(v.quantile(0.95))

df.groupby("k").agg(p95("value").alias("p95"))
```

Caching y persistencia

```
from pyspark import StorageLevel

df_small = (df
    .filter("dt >= '2025-01-01'")
    .select("k", "v")
    .persist(StorageLevel.MEMORY_AND_DISK_SER))

_ = df_small.count() # materializa el cache
```

Rutas reales (Databricks / Azure ADLS)

```
# DBFS (Databricks)
dbfs:/mnt/bronze/events/2025/10/01/
dbfs:/mnt/silver/sales/
dbfs:/mnt/gold/sales_by_customer/

# ABFS (ADLS Gen2) montado en Databricks
abfss://bronze@mydatalake.dfs.core.windows.net/events/2025/10/01/
abfss://gold@mydatalake.dfs.core.windows.net/sales_by_customer/

# Lectura/escritura Delta
spark.read.format("delta").load("dbfs:/mnt/silver/sales")

(df.repartition(256)
    .write
    .format("delta")
    .mode("overwrite"))
```

```
.option("maxRecordsPerFile", 2_000_000)
.save("dbfs:/mnt/gold/sales_by_customer"))
```

8. Delta Lake: OPTIMIZE, Z-ORDER, VACUUM, ANALYZE

Compaction (OPTIMIZE)

```
OPTIMIZE sales;
```

Z-ORDER (data skipping fino)

```
OPTIMIZE sales ZORDER BY (customer_id, order_date);
```

VACUUM y ANALYZE TABLE

```
VACUUM sales RETAIN 168 HOURS;
ANALYZE TABLE sales COMPUTE STATISTICS;
```

Cuándo usar Z-ORDER

- Cuando tienes filtros frecuentes por columnas de alta cardinalidad (id, fechas).
- Cuando quieres acelerar queries analíticas con WHERE por esas columnas.
- No aporta en full scans o tablas muy temporales.

9. Photon vs JVM/GC

Photon (en Databricks) ejecuta el motor de ejecución en C++ con vectorización completa, reduciendo el impacto del GC y acelerando joins y aggregations CPU-bound. Con la JVM clásica, conviene usar G1GC y suficiente memoryOverhead para evitar paradas de GC y OOM.

```
# Configuración típica de cluster Databricks
spark.databricks.photon.enabled = true
spark.executor.cores = 5
spark.executor.memory = 8g
spark.executor.memoryOverhead = 2g
```

10. Autoscaling vs Single Node

Autoscaling merece la pena cuando los jobs duran muchos minutos u horas y el workload varía. Para jobs cortos (1–3 minutos), el tiempo de escalar nodos puede ser mayor que el beneficio, y el coste sube.

Single Node es ideal para compactaciones (OPTIMIZE/VACUUM), pruebas y ETL pequeño (< 50 GB).

11. AQE (Adaptive Query Execution)

AQE permite que Spark adapte el plan físico en tiempo de ejecución: coalesce de particiones, manejo de skew y cambio dinámico del tipo de join (por ejemplo a broadcast cuando una tabla es más pequeña de lo esperado).

```
SET spark.sql.adaptive.enabled = true;
SET spark.sql.adaptive.coalescePartitions.enabled = true;
SET spark.sql.adaptive.skewJoin.enabled = true;
SET spark.sql.adaptive.shuffle.targetPostShuffleInputSize = 134217728; -- 128 MB
```

12. Playbooks y Checklists

Playbook: Job lento

- EXPLAIN FORMATTED → revisa tipo de join y ordenaciones.
- Spark UI → detecta skew, spill y tareas outlier.
- Activa/ajusta AQE y spark.sql.shuffle.partitions.
- Usa broadcast en el lado pequeño del join.
- Aplica filtros y proyecciones antes del join/agg.
- Reparticiona por la clave caliente.
- Si sigue mal: salting + pre-agregaciones.

Playbook: OOM / GC alto

- Reduce tamaño de partición aumentando el número de shuffle.partitions.
- Sube spark.executor.memoryOverhead y usa MEMORY_AND_DISK_SER.
- Evita UDFs Python; usa funciones nativas o pandas UDF.
- Lee/escribe en Delta con sólo las columnas necesarias.

Playbook: Millones de archivos pequeños

- Reescribe la tabla con repartition(n) y/o maxRecordsPerFile.
- En Delta, ejecuta OPTIMIZE (compaction) y considera Z-ORDER si aplica.

13. Anexo: Chuleta Spark Master (comandos y parámetros)

```
-- AQE y Skew
SET spark.sql.adaptive.enabled = true;
SET spark.sql.adaptive.coalescePartitions.enabled = true;
SET spark.sql.adaptive.skewJoin.enabled = true;

-- Shuffle & Files
SET spark.sql.shuffle.partitions = 192;
SET spark.sql.files.maxPartitionBytes = 134217728; -- 128 MB

-- Broadcast
SET spark.sql.autoBroadcastJoinThreshold = 200MB;
SET spark.sql.broadcastTimeout = 600;

-- Arrow (PySpark vectorizado)
SET spark.sql.execution.arrow.pyspark.enabled = true;

-- Photon (Databricks)
SET spark.databricks.photon.enabled = true;

# PySpark: patrones comunes
df = (spark.read.format("delta")
      .load("dbfs:/mnt/bronze/fact_sales")
      .select("k", "ts", "amount")
      .filter("ts >= '2025-09-01'"))

df = df.repartition(192, "k")
res = df.join(F.broadcast(dim), "k")

# Escribir Delta con control de archivos
(df.repartition(256)
 .write
 .option("maxRecordsPerFile", 2_000_000)
 .format("delta")
 .mode("overwrite")
 .save("dbfs:/mnt/gold/sales_by_customer"))
```